TIPS & TRICKS

File permissions with access control lists (ACLs)

ACLs can be used on an Ext3 filesystem to manage file permissions. ACLs expand the basic read, write and execute permissions to more categories of users and groups than can be done using *chmod*.

In addition to managing permissions for the owner of the file and the group, ACLs allow permissions to be set for any user or user group(s), and even for those users who are not part of any group.

Consider a situation where you want to grant write permission only to one user from a group. If you set the permission from 'chmod', all other users from the group will get write access on the file. In such a situation, using ACLs will work.

The example below will help you understand better.

Assume that we have created three users—Bhargav, Parishesh and Hardik. All three are members of the Apache group and the users' primary group is also Apache.

Now, create a '/example' directory and change the ownership to 'bhargav' and group name to 'apache'.

```
[root@aum42 ~]# mkdir /example
[root@aum42 ~]# ll -d /example/
drwxr-xr-x 2 root root 4096 2010-11-13 11:38 /example/
[root@aum42 ~]# chown bhargav:apache /example
[root@aum42 ~]# ll -d /example/
drwxr-xr-x 2 bhargav apache 4096 2010-11-13 11:38 /example/
```

Even though 'Bhargav' wishes to grant write permission only to Parishesh, Hardik will also get write access if Bhargav sets write permission on the 'apache' group. So Bhargav can use ACL to grant exclusive write access to Parishesh.

```
[bhargav@aum42 ~]$ setfacl -m u:parishesh:rwx /example/
```

To view the ACLs for the '/example' directory, use the command given below:

```
[bhargav@aum42 ~]$ cd /
[bhargav@aum42 /]$ getfacl example/
# file: example
# owner: bhargav
# group: apache
user::rwx
user:parishesh:rwx
group::r-x
mask::rwx
other::r-x
```

Now, only Bhargav (the owner) and Parishesh will have full permission over the '/example' directory. To verify this, log in with 'parishesh' as the user on the other terminal.

```
[parishesh@aum42 ~]$ cd /example/
```

Next, try to create a directory named 'test'.

```
[parishesh@aum42 example]$ mkdir test
```

Go to the other terminal, log in with 'hardik' as the user and try to make the test2 directory in the '/example' directory.

```
[hardik@aum42 ~]$ cd /example/
[hardik@aum42 example]$ mkdir test2
mkdir: cannot create directory `test2\': Permission denied
```

To remove all extended ACL entries, use the following command:

```
[bhargav@aum42 ~]$ setfacl -b /example
```

—Bhargav Trivedi,
bhargav@aspl.in

Secure your login without using a password every time

There are several ways to do a password-less login through 'ssh'. Given below are a few steps that will help you set up one such environment in which you do not need to enter your password every time you try to log in using 'ssh'.

1. On the client, run the following commands:

```
$ mkdir -p $HOME/.ssh
$ chmod 0700 $HOME/.ssh
$ ssh-keygen -t rsa -f $HOME/.ssh/id_rsa -P ‘’
```

This should result in two files, '$HOME/.ssh/id_rsa' (private key) and '$HOME/.ssh/id_rsa.pub' (public key).